next-workflow 2.0: Architecture Review

A second independent review after the v2 fixes: what works, what's still risky, and what to improve next.

Scope: the whole kit at v2.0.0. That's 8 commands, 8 agents, 3 skills with 18 references, 13 hook and helper scripts, the installer, settings, templates, the eval harness and the self-test (189 assertions). Reviewed 15 Oct 2026. Findings marked ✓ **VERIFIED** were reproduced by running the code. `LIKELY` means inferred from the code but not executed. This review covers changes I made myself in v2, and holds them to the same standard as everything else.

1. Verdict

3.7

out of 5 (v1: 3.4)

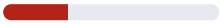
The architecture is now right, but it hasn't been proven to work.

v2 has the shape a serious agent workflow should have. You approve the plan before anything is built. Each task runs in a fresh session. An independent agent checks the result. Every task can be rolled back. And the kit now records time, cost and check hits. All 18 v1 findings are fixed and covered by tests.

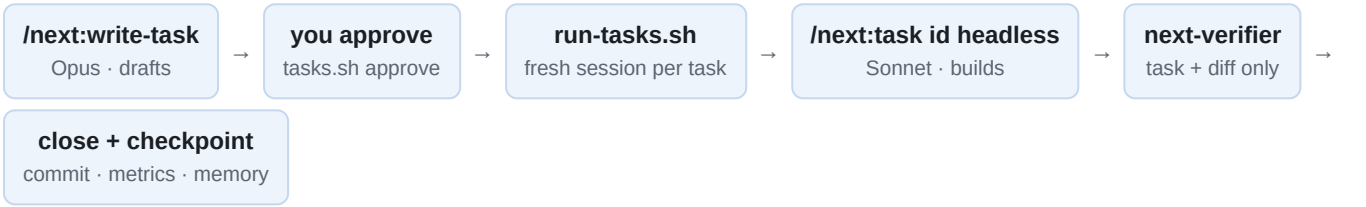
Two things hold the score back. First, **the full loop has never actually run**: the eval harness exists but has never been executed, so we don't know how well the tasks come out. Second, **v2 introduced new sharp edges**, the worst being that a checkpoint commit can capture a `.env` file. Both are cheap to fix compared with what's already built.

Scorecard: v1 → v2

Dimension	v1	v2		Why
Architecture & flow	4.0	4.5		Plan → approve → build → verify → checkpoint is the right loop.
Token efficiency	4.5	4.0		Fresh sessions help; the verifier adds a subagent to almost every task.
Code-quality guardrails	4.0	3.5		Shell writes are now checked, but that check also blocks over problems that were already there (N4).
Security	2.5	3.5		H1 and H2 are fixed; the checkpoint can commit secrets (N1); the secrets guard is easy to get around (N5).
Reliability & recovery	2.5	4.0		Fresh context, checkpoints, undo and resumable batches. Undo ignores dependent tasks (N6).
Verification depth	3.0	3.5		There's an independent verifier, but the browser path probably doesn't work yet (N3).

Framework currency	2.5	3.5		Next 16 is covered, but written from memory and not checked against the docs (N8).
Maintainability	3.5	3.0		Grew from 8 to 13 scripts; the kit itself isn't in git and has no CI (N9).
Observability	1.5	3.0		Metrics and stats now exist; the most important signal, verifier verdicts, isn't recorded (N7).
Proven outcomes	n/a	1.5		The eval has never run; the headless loop has never been exercised end to end (N2).

2. How v2 fits together



The hooks run underneath: `guard-secrets` before shell commands; `post-edit` (the guards plus the formatter) after edits; `bash-guard` after shell commands; and `verify` (typecheck, failing only on new errors) at the end of each turn. `evals/run.sh` is meant to measure the whole loop on a fixture app.

3. The good side

A human approves the plan

Drafts can't be built until you approve them. Planning on Opus, approval by you and building on Sonnet is the right split of cost and responsibility.

A fresh session per task

`run-tasks.sh` removes context build-up, the biggest quality risk in v1. The project memory file is the only thing carried between tasks, as designed.

An independent verifier

It sees only the task and the diff, and each criterion is tagged with how it must be proven. "Never PASS a `[browser]` item from reading code" is exactly the right rule.

Recovery

Per-task checkpoint commits, `undo`, batches that resume where they stopped, and a two-round fix limit before a task is blocked. Failures are bounded and reversible.

Honest verification

The type-error baseline stops the agent from being bounced for old errors or wandering into fixing them, and the agent is told not to baseline its own errors.

Hardened defaults

Stack values are sanitised before use, a tampered cache is rejected, the `/home` read rule is gone, `~/.ssh` and `~/.aws` are denied, and auditors are read-only.

Version-aware

The installed Next major and Tailwind version decide which rules apply. Next 16 projects get `proxy.ts` and `'use cache'` guidance; Next 15 projects aren't told about it.

Well-tested plumbing

189 assertions, including a regression test for every v1 finding. Rerunning them caught three bugs during v2 development: invalid JSON ids, undo deleting the task file, and hook entries being reordered.

The parts to measure itself are in place

`metrics.jsonl`, `tasks.sh stats`, cost per task from headless runs, and an eval harness with objective checks. Nobody has used them yet, but they exist.

Clean lifecycle

A version stamp, upgrades that replace old hook entries instead of stacking them, and an uninstall that keeps your tasks, memory and own hooks.

4. The bad side: findings

Numbered N1–N15 to keep them apart from the v1 findings. HIGH fix before relying on it · MEDIUM fix soon · LOW polish.

N1 A checkpoint commit can capture secrets ✓ VERIFIED

Where `hooks/tasks.sh` `close`: `git add -A` before the checkpoint commit.

Impact Any untracked file that isn't git-ignored goes into the commit. In my test, an untracked `.env` was committed. The `start` warning about a dirty tree goes to the model, not to you, so in a headless batch nobody sees it.

Fix Stage only the paths in `tasks.sh diff <id>` (the task's own changes), and refuse to checkpoint if any staged path matches `.env*`, `*.pem` or a key file. When `--checkpoint` is used, have `run-tasks.sh` create a branch (`nw/batch-<date>`) so commits never land on `main`.

N2 The full loop has never run

Where `evals/run.sh` has only been dry-run; there's no `evals/results/`. `claude -p "/next:task <id> headless"`, the verifier inside a headless session, and cost capture have never been exercised.

Impact Everything else in this report is about structure. We don't know the pass rate, the cost per task, or whether headless runs get stuck on denied permissions. A workflow that hasn't been measured is a hypothesis.

Fix Run `bash evals/run.sh` three times on v2.0.0 and keep the results as the baseline. From then on, compare every prompt change against it.

N3 Browser verification probably can't run LIKELY

Where `agents/next-verifier.md`: "write a throwaway spec in `.claude/.cache/verify/` … run it with `npx playwright test <that file>`".

Impact There are three obstacles:

- Playwright only finds tests inside the config's `testDir`, so a spec under `.claude/.cache` is likely "No tests found".
- The verifier has no Write tool, so it would have to write the spec through the shell.
- In headless `acceptEdits` runs, that shell command isn't allowed and is denied.

So `[browser]` criteria would always end up UNVERIFIED in batches.

Fix Ship `.claude/verify.playwright.config.ts` with `testDir: '.cache/verify'` and `baseURL` taken from `NEXT_WF_DEV_URL`. Allow `Bash(npx playwright test -c .claude/verify.playwright.config.ts:*)`, and give the verifier Write access limited to that folder by instruction. Add a test that runs it on the eval fixture.

N4 The shell check blocks over problems that were already there ✓ VERIFIED

Where `hooks/bash-guard.sh` runs the full guards, including the blocking ones, on every file a shell command changed.

Impact When the agent runs `prettier --write .`, a codemod or `git stash pop`, every touched file with an *old* violation blocks the turn (exit 2). The agent then either "fixes" unrelated code or fights the hook. It's the M1 problem again, in a new place.

Fix For shell-changed files, block only on violations that aren't in the `HEAD` version (run the guard on `git show HEAD:<file>` and compare). Or simply make shell-path findings advisory when more than ~5 files changed.

N5 The secrets guard blocks normal commands, is easy to get around, and contradicts itself ✓ VERIFIED

Where `hooks/guard-secrets.sh`; `settings.json` deny `Read(./.env.*)`.

Impact `echo ".env" >> .gitignore` and `cp .env.example .env.local` are blocked, but `cat .e""nv` gets through. The block message says "read .env.example", yet the Read deny rule `./.env.*` also denies `.env.example`.

Fix Narrow the Read deny rules to the real secret files: `.env`, `.env.local`, `.env.*.local`, `.env.production`. Allow commands that only write to `.gitignore`. Describe the guard as a speed bump: the real protection is keeping secrets out of the workspace, or running batches in a container.

N6 Undo ignores dependent tasks

Where `tasks.sh undo` reverts one task and sets it back to `todo`.

Impact Undoing task 003 while 004 (`depends: 003`) stays `done` leaves code that relies on what was just reverted, or a revert conflict halfway through.

Fix Refuse when a `done` task depends on the target, and list those tasks. Add `undo --cascade` to revert the dependents first, newest first.

N7 Verifier verdicts aren't recorded ✓ VERIFIED

Where `metrics.jsonl` has events for task start/close, guard hits and runs, but none for the verifier.

Impact The single best quality signal (how often a task passes on the first try, and which kinds of criteria fail) is lost. You can't tell whether Sonnet is good enough, or whether the planner writes criteria that are too hard to check.

Fix Add `tasks.sh verdict <id> PASS|FAIL|UNVERIFIED <round>`, have the runner call it after each verifier round, and show the first-pass rate and failures by tag in `stats`.

N8 The Next 16 rules are unchecked against the docs

Where `skills/nextjs-app-router/references/next16.md`, written from memory: the `revalidateTag` profile argument, `'use cache: private'`, `refresh()`, Turbopack and `--webpack`.

Impact Wrong guidance that sounds confident is worse than none, because the agent will follow it, and guard advisories repeat it.

Fix Check each rule against the official Next.js 16 upgrade guide and caching docs. Add a "verified against Next X.Y on <date>" line and a source link per section, and recheck on each Next minor release.

N9 The kit itself has no version control or CI ✓ VERIFIED

Where The kit folder isn't a git repository; `test.sh` runs only by hand.

Impact There's a `VERSION` file but no history behind it: you can't diff v1 against v2, bisect a regression, or tie an eval result to an exact prompt version.

Fix Put the kit in git, tag `v2.0.0`, and keep a short CHANGELOG. Add CI that runs `./test.sh` and `shellcheck` on every change.

N10 The eval is too thin to trust one run

Where `evals/`: 4 tasks, one run, grep-based checks, a Next 15 fixture only.

Impact Model output varies, so one run can swing ±25% on 4 tasks. The grep checks are brittle (for example, check 003 requires the literal text `ownerId`), so a correct solution named differently fails. Nothing covers Next 16.

Fix Run each eval 3 or more times and report the mean and spread. Replace grep checks with behaviour checks: hidden Vitest specs the agent never sees, plus requests against the built app. Add a Next 16 + `cacheComponents` fixture and 2–3 tasks that are deliberately ambiguous, to test blocking.

N11 The rule for skipping the verifier is subjective

Where `task.md`: "the diff is a few lines".

Fix Make it mechanical: skip only if every criterion is `[review]` and `tasks.sh diff --stat` shows 20 changed lines or fewer. Record the skip as a metric.

N12 The memory file is cut off without warning ✓ VERIFIED

Where `session-context.sh` uses `head -c 4000`.

Fix When the file is over the limit, add a line saying "project-context.md truncated, prune it", so the oldest facts (often the Gotchas section) aren't dropped without anyone knowing.

N13 **Complexity and per-edit delay are growing** ✓ VERIFIED

Where 13 scripts. `post-edit` takes about 0.65 s per edit *before* Prettier or ESLint run (measured), and `git status` runs after every shell command.

Fix Fine for now. Keep an eye on it with `stats`. If the per-edit delay goes past about 2 s on real repos, move the guards into one Node script that parses the file once.

N14 **Global install is second-class**

Where The permission rules list `bash .claude/hooks/tasks.sh` only; the commands rely on a sentence telling Claude to use `~/.claude/hooks` when the local path is missing.

Fix Have the session report state the exact helper path; in `--global` mode, write matching permission rules; add a test of a full task cycle under a global install.

N15 **Some limits rely on the model obeying**

Where The in-session batch limit of 3, "never baseline your own errors", and "edit only with Edit/Write" are instructions with only partial enforcement behind them.

Fix Accept these as soft limits, but measure them. Record each in-session batch length and each `bash-guard` hit in `stats`; if they're regularly broken, enforce them in `tasks.sh` (for example, a per-session counter).

5. Improvement roadmap

NOW **Safety and truth: about one day**

- **N1**: checkpoint only the task's own paths, refuse secret files, use a batch branch.
- **N3**: a kit Playwright config and allow rule, so `[browser]` checks can actually run.
- **N5**: narrow the Read deny rules and unblock writes to `.gitignore`.
- **N11, N12**: a mechanical skip rule and a truncation warning.

NEXT **Prove it: about one week**

- **N9**: put the kit in git, tag v2.0.0, add CI for `test.sh` and shellcheck.
- **N2**: run the eval three times and commit the results as the baseline.
- **N7**: record verifier verdicts and show the first-pass rate.
- **N4, N6**: shell-check blocks only on new violations; dependency-aware undo.

LATER **Make it robust**

- **N10**: behaviour-based checks, multiple runs, a Next 16 fixture, ambiguous tasks.
- **N8**: Next 16 rules checked against the docs, with sources and a recheck date.
- **N13, N14, N15**: watch the per-edit delay, make global install first-class, measure the soft limits.
- Once there's data: decide whether simple tasks can drop to Haiku, using first-pass rate and cost per task from `stats`, not guesswork.

> **If you do only three things:** fix N1 (secrets in commits), make browser verification work (N3), and run the eval three times (N2). After that, the workflow isn't just well designed: you'll know how well it performs and what it costs.

6. Appendix: how the verified findings were reproduced

N1: checkpoint commits an untracked .env

```
echo "SECRET=1" > .env
NEXT_WF_CHECKPOINT=commit tasks.sh close 001 x
git show --stat HEAD   →   .env | 1 +
```

N4: shell reformat blocked over an old violation

```
app/old.tsx (committed): 'use client' + export default async function
bash-guard --snapshot; sed -i (whitespace only) app/old.tsx
bash-guard ← "npx prettier --write ."   →   exit 2 (blocked)
```

N5: secrets guard

```
echo ".env" >> .gitignore      → rc=2 (blocked, legitimate)
cp .env.example .env.local     → rc=2 (blocked, legitimate)
cat .e""nv                     → rc=0 (passes, bypass)
```

N7, N12, N13

```
grep -c verifier .claude/.cache/metrics.jsonl  → 0
session-context.sh:37   … | head -c 4000
post-edit.sh, one .tsx edit, no node_modules  → 0.65 s
```